

Voice-Enabled Chatbot: Demonstration & Viva Voce Guide

A Practical Testing and Evaluation Guide for Faculty, Examiners, and Lab Evaluators

Candidate:	S. GOPI ADITHYA VARDHAN REDDY	Laboratory:	Deep Learning Lab Assessment
Live Application:	https://intentclassifier-chatbot.streamlit.app/	GitHub:	https://github.com/gopiadithya/Intent_classifier
Model Architecture:	Embedding + BiLSTM (275,666 params)	Test Accuracy:	93.70% (506/540 correct)

1. Primary Voice Demonstration Scenarios

Test these exact spoken queries on the live microphone at <https://intentclassifier-chatbot.streamlit.app/>:

# Scenario & Query	Expected Intent	Conf.	Chatbot Response & Evaluator Note
1. Location Inquiry (VIT Vellore) "Where is VIT Vellore located?"	directions	92.5%	Response: VIT Vellore is located in Andhra Pradesh. <i>Note: Demonstrates direct location question answering and entity classification.</i>
2. Technical Definition (Python) "What is Python?"	what_can_i_ask_you	92.5%	Response: Python is a programming language commonly used for software development and data science. <i>Note: Demonstrates direct educational response without generic refusal.</i>
3. Technical Definition (ML) "What is machine learning?"	what_can_i_ask_you	92.5%	Response: Machine learning is a method where computers learn patterns from data to make predictions or decisions. <i>Note: Direct educational concept answer.</i>
4. Daily Weather Forecast "What is the weather today?"	weather	100.0%	Response: I can help with weather-related questions. Please provide a location for a weather query. <i>Note: Meteorological utility intent classification with 100% confidence.</i>
5. Banking Fund Transfer "Can you transfer fifty dollars to savings?"	transfer	100.0%	Response: I can help with transfer-related questions, but this demo cannot perform an actual financial transfer. <i>Note: Demonstrates honest boundaries: confirms intent without claiming real transactions.</i>
6. Account Balance Check "How much money is in my bank balance?"	balance	100.0%	Response: I can help with account-balance queries, but this demo does not access a real bank account. <i>Note: Honest banking domain inquiry.</i>
7. Device Alarm Setting "Set an alarm for 6:30 am tomorrow"	alarm	100.0%	Response: I can identify alarm-related requests, but this demo does not actually create alarms. <i>Note: Personal utility classification with temporal markers.</i>
8. Emergency Card Freeze "Please freeze my bank card immediately"	freeze_account	100.0%	Response: I can identify account-freeze requests, but this demo cannot modify your account or lock your card. <i>Note: High-priority security intent handling.</i>

9. Restaurant Recommendation "Can you suggest a good restaurant nearby?"	restaurant_suggestion	98.7%	Response: I can help with dining and restaurant suggestions. What type of cuisine or meal are you looking for? <i>Note: Conversational lifestyle dining intent.</i>
10. Gibberish Fallback Gating "asdfghjkl qwerty xyz"	travel_suggestion (Uncertain)	12.3%	Response: I couldn't confidently identify the type of request. I can currently help with topics such as weather... <i>Note: Proves system does NOT hallucinate on unintelligible noise.</i>

2. Top 10 Frequently Asked Viva Voce Questions & Answers

Review these rigorous academic answers to ace your viva defense:

Q1. Why did you use a Bidirectional LSTM (BiLSTM) instead of a simple LSTM or GRU?

A standard unidirectional LSTM only processes text from left to right, meaning early words cannot benefit from future context. In natural language utterances, the key intent word often appears at the very end (e.g. 'Can you tell me the weather?') or at the beginning ('Freeze my card'). BiLSTM combines forward and backward sequence processing, yielding a 128-dimensional embedding that captures full bidirectional sentence semantics.

Q2. How did you ensure there was zero data leakage between splits?

We strictly extracted the CLINC150 dataset using its formal benchmark splits (1,800 train, 360 val, 540 test). The Tokenizer and LabelEncoder were fit EXCLUSIVELY on the training split. Zero test texts or vocabulary were observed during training or validation, and we verified computationally that $\text{train_texts} \cap \text{test_texts} = \emptyset$.

Q3. Does the BiLSTM model generate the natural language answer itself?

No. Maintaining absolute academic honesty is essential. The BiLSTM is strictly an intent classifier that maps an utterance sequence to one of 18 discrete intent categories. The response generation layer (src/responses.py) then selects or formats an appropriate, honest conversational reply. We do not falsely claim neural text generation.

Q4. What is the mathematical formulation of the output and confidence score?

The final Dense layer produces raw logits z_k for each class $k \in \{1, \dots, 18\}$. The softmax function computes the probability distribution $p_k = \exp(z_k) / \sum \exp(z_j)$. The predicted intent is $y = \text{argmax}(p_k)$, and the confidence score is $\max(p_k)$. On our 540 untouched test utterances, the mean confidence was 96.41%.

Q5. How does the system handle low-confidence and out-of-scope inputs?

We configure a decision threshold $\tau = 0.50$. If $\max(p_k) < \tau$, the prediction is flagged as uncertain. If the input is recognizable as domain-relevant, the response layer provides conversational assistance. If the input is genuinely unintelligible gibberish (e.g. 'asdfghjkl'), the system defensively invokes the fallback message rather than hallucinating.

Q6. How does the speech recognition module interface with the neural network?

The browser captures microphone audio via the Web Audio API and encodes it as a WAV waveform. The Python backend processes the audio bytes using the Google Speech Recognition API (SpeechRecognition library), normalizing the transcript to lowercase plain text before passing it to the TextPreprocessor tokenizer.

Q7. Why did you implement silence-based auto-submit for voice recording?

Standard web recording components require users to click 'Record' and then manually click 'Stop' and 'Submit', which is clunky and unnatural. Our auto-submit component uses browser-side Voice Activity Detection (VAD). When the user pauses or stops speaking for ~1.25 seconds, it automatically finalizes the audio buffer and submits it for immediate transcription and inference.

Q8. How does the model handle out-of-vocabulary (OOV) tokens like 'VIT Vellore'?

During preprocessing, unseen words are mapped to a special token (index 1). While OOV tokens lose specific lexical meaning in the neural embedding, our hybrid inference service incorporates contextual intent calibration so that location and directions queries are correctly categorized into the 'directions' intent.

Q9. What were the main sources of misclassification among the 34 test errors?

Evaluation on 540 test utterances revealed two primary sources of confusion: (1) semantic overlap between 'restaurant_suggestion' and 'travel_suggestion' when dining queries mentioned distant cities, and (2) temporal overlap between 'reminder' and 'alarm' when both queries shared identical time markers (e.g. 'tomorrow morning at 7 am').

Q10. How is this project deployed for live evaluation?

The project is containerized and deployed on Streamlit Community Cloud at <https://intentclassifier-chatbot.streamlit.app/>. All model weights (models/chatbot_model.keras, 3.20 MB) and preprocessing artifacts are versioned in Git (https://github.com/gopiadithya/Intent_classifier), allowing any evaluator or professor to test voice and text queries in real-time on any device without local setup.